Elementor Framework Forge

Phase 2 — Colors Variables Edit Space

Claude Code Build Specification — v2.0

Jim Roberts — Jim R Forge — jimrforge.com

Companion to: EFF-Claude-Code-Spec-v2.docx

1. Overview

This document specifies the Colors Variables edit space — the first content area built for the EFF Center Edit Space (Phase 2). It is an addendum to EFF-Claude-Code-Spec-v2.docx and assumes that the Phase 1 framework (panels, navigation, modal system, CSS parser, data store) is fully in place.

When the user selects the Colors node or any Colors subgroup (category) in the left menu panel, the Center Edit Space renders the Colors page. This page displays all Elementor v4 color variables organized into user-definable categories.

> **Spec amendment:** *"Status" category renamed to "Semantic" throughout. New default category "Uncategorized" added. See Section 2.1.*
>
> **Architecture decision:** *EFF operates as a staging layer between the user and Elementor. All edits are made in EFF's own data store first. Changes are pushed to Elementor only when the user explicitly presses "Commit to Elementor" (a new distinct action from "Save EFF File"). See Section 11 for the full two-store architecture discussion.*

2. Category System

Color variables are organized into named categories. Categories are stacked vertically in the edit space. Each category contains zero or more color variable rows. The category set, category names, and their order are stored in the EFF data store and mirrored in the left menu panel under the Colors node.

2.1 Default Categories

EFF ships with five default categories, pre-populated in eff-defaults.json. Default categories have no special properties — they may be renamed, reordered, duplicated, or deleted exactly like user-created categories. The single constraint is that the Uncategorized category must always exist and cannot be deleted.

Order	Category Name	Purpose
1	Branding	Primary brand colors — logo, primary palette
2	Background	Page, section, and container background colors
3	Neutral	Grays, off-whites, dark neutrals
4	Semantic	Status/meaning colors — success, error, warning, info (formerly "Status")
5	Uncategorized	Default landing category for color variables new to EFF. Cannot be deleted.

> **eff-defaults.json update required:** *Change "Status" to "Semantic" and add "Uncategorized" as the fifth entry in the Colors category array.*

2.2 Category Data Model

```
{
  "id":          "uuid-v4",
  "name":        "Branding",
  "group":       "Variables",
  "subgroup":    "Colors",
```

```
  "order":       1,
  "expanded":    true,
  "created_at":  "ISO8601",
  "updated_at":  "ISO8601"
}
```

expanded — Boolean. Persisted per-category. Controls whether color rows are visible. Default: true.

2.3 Left Menu Sync

- When a category is added in the edit space, it appears in the left menu immediately
- When a category is renamed, the left menu label updates immediately
- When categories are reordered (Move Up / Move Down), the left menu order updates immediately
- When a category is deleted, it is removed from the left menu immediately
- The currently expanded category in the edit space does not automatically change the left menu selection — they are independent

3. Colors Page Layout

The Colors page fills the entire Center Edit Space. It is a vertically scrollable list of category blocks stacked top-to-bottom with no gaps — a continuous stack.

```
┌──────────────────────────────────────────────────────────────────┐
│  CENTER EDIT SPACE — Colors Page                                 │
├──────────────────────────────────────────────────────────────────┤
│  [Filter/search bar_________________]  [Collapse all] [Expand all]│
│  ┌──────────────────────────────────────────────────────────┐    │
│  │ CATEGORY HEADER: Branding [dup][▲][▼][cpy][pst][+][🗑][ᵥ] │    │
│  ├──────────────────────────────────────────────────────────┤    │
│  │ [swatch] [name................] [value.......] [fmt▼] [ᵥ] │    │
│  │ [swatch] [name................] [value.......] [fmt▼] [ᵥ] │    │
│  └──────────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │ CATEGORY HEADER: Neutral   [dup][▲][▼][cpy][pst][+][🗑][▶] │    │
│  │                            (collapsed — no rows shown)    │    │
│  └──────────────────────────────────────────────────────────┘    │
│  [ ... Semantic ... ]   [ ... Uncategorized ... ]                │
└──────────────────────────────────────────────────────────────────┘
```

4. Category Header

Each category opens with a full-width header bar, always visible even when collapsed.

4.1 Header Layout

```
┌──────────────────────────────────────────────────────────────────────────┐
│  ● CATEGORY NAME            [Dup] [▲] [▼] [Copy] [Paste] [+] [Del] [v]   │
└──────────────────────────────────────────────────────────────────────────┘
  Left:  gold dot + editable category name
  Right: icon buttons (Section 4.2) + Expand/Collapse toggle (always last)
```

Background: --eff-bg-panel (#faf9f6). Bold name. Gold dot (--eff-clr-accent). Height: ~40px. border-radius: 6px 6px 0 0 expanded, 6px collapsed.

4.2 Category Header Icon Buttons

All buttons follow .eff-icon-btn style (main spec Section 8.2). Right-aligned, fixed order.

Pos	Icon	Tooltip	Action
1	Duplicate (two squares)	Duplicate category	Copies category + all color assignments immediately below. New name appended with " — Copy".
2	Arrow up (▲)	Move category up	Moves category one position up. Disabled when first.
3	Arrow down (▼)	Move category down	Moves category one position down. Disabled when last.
4	Copy (clipboard)	Copy category	Copies category definition + color assignments to EFF internal clipboard. Does not affect OS clipboard.
5	Paste (clipboard+arrow)	Paste colors	Pastes colors from EFF clipboard into this category. Disabled if clipboard empty.
6	Plus (+)	Add color variable	Appends a new empty color row to this category.
7	Trash (🗑)	Delete category	Deletes category. If it contains colors, confirms: "Delete [name]? Its [n] colors will be moved to Uncategorized." Cannot delete Uncategorized.
8 (last)	Chevron v / ^	Collapse / Expand	v = currently collapsed, click to expand. ^ = currently expanded, click to collapse. Always rightmost button.

> **Disabled state:** *Move Up disabled when first. Move Down disabled when last. Paste disabled when clipboard empty. Disabled buttons are visually muted (--eff-clr-muted, pointer-events: none).*

4.3 Category Name — Inline Editing

- Default display: bold text label at --fs-md (16px)
- Single-click activates edit mode — label becomes an input field with visible border
- Enter or click outside commits the change

- Escape cancels and restores the previous name
- Empty names are not permitted — previous name restored on empty commit
- Name change: update EFF data store, update left menu label. Does not set hasEFFUnsavedChanges unless the EFF file is being tracked (see Section 11.2)

5. Color Variable Row

Each color variable within a category is displayed as a full-width horizontal row.

5.1 Row Layout

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ [Swatch] [Name field──────────────────] [Value field──────] [Fmt▼] [v] │
└──────────────────────────────────────────────────────────────────────────────┘

  Swatch:          ~48px wide, ~36px tall, border-radius 8px
  Name field:      flex-fill — fills all remaining width
  Value field:     ~180px fixed
  Format selector: ~90px fixed (HEX / HEXA / RGB / RGBA / HSL / HSLA)
  Expand button:   ~32px fixed — opens shades/tints + color picker below
```

Row background: alternating --eff-bg-card (#fff) and --eff-bg-panel (#faf9f6). Row height: ~44px collapsed. Border-bottom: 1px solid --eff-clr-border.

5.2 Field Border & Shadow Formatting

All fields in the color variable row use a pit / recess shadow treatment. Each field reads as a rectangular well pressed into the page surface, lit from overhead — dark shadow on the near edges (top and left), lighter on the far edges (bottom and right). The color swatch uses a deeper vessel treatment: a lake filled with color, with no outer raise.

> **Approved:** *Variant A — Subtle Pit. Confirmed by Jim Roberts. CSS tokens below are final and ready for eff-theme.css.*

Shadow logic — pit / recess:

```
/* The pit illusion uses two shadow layers working together:        */
/*                                                                   */
/* OUTSIDE the element (outer box-shadow):                          */
/*   dark offset negative (up-left)  = near rim blocked from light */
/*   warm offset positive (down-right) = far rim catches light     */
/*                                                                   */
/* INSIDE the element (inset box-shadow):                           */
/*   dark inset top + left   = near walls in shadow                 */
/*   light inset bottom + right = far walls catching light          */
```

Text fields and format selector — subtle pit (Variant A, approved):

```
/* Pit field tokens — defined in eff-theme.css */
--eff-field-shadow:
  /* outer: dark near rim (top-left) */
  -1px -2px  4px rgba( 61,  47,  31, 0.25),
  /* outer: warm far rim (bottom-right) */
   1px  2px  3px rgba(255, 245, 220, 0.55),
  /* inner: dark near walls (top, left) */
  inset  0    4px  7px rgba( 61,  47,  31, 0.16),
  inset  4px  0    7px rgba( 61,  47,  31, 0.10),
  /* inner: light far walls (bottom, right) */
  inset  0   -3px  5px rgba(255, 255, 255, 0.55),
  inset -3px  0    5px rgba(255, 255, 255, 0.35);

.eff-color-field,
.eff-color-format {
  border:     1px solid var(--eff-clr-primary);   /* #3d2f1f brown */
```

```
  box-shadow: var(--eff-field-shadow);
}

.eff-color-field:focus {
  border-color: var(--eff-clr-accent);            /* #f4c542 gold  */
  box-shadow:
    var(--eff-field-shadow),
    0 0 0  3px rgba(244, 197,  66, 0.38),         /* gold glow ring */
    0 0 9px rgba(244, 197,  66, 0.22);
}
```

Color swatch — lake / filled vessel (Variant A, approved):

```
/* Swatch lake tokens — inset-only, heavier on top and left */
--eff-swatch-shadow:
  inset  0    5px 10px rgba(0, 0, 0, 0.26),      /* top wall       */
  inset  5px  0   10px rgba(0, 0, 0, 0.18),      /* left wall      */
  inset  0   -4px  7px rgba(0, 0, 0, 0.12),      /* bottom wall    */
  inset -4px  0    7px rgba(0, 0, 0, 0.09);      /* right wall     */

.eff-color-swatch {
  border:     1px solid var(--eff-clr-primary);   /* #3d2f1f brown */
  box-shadow: var(--eff-swatch-shadow);
}
```

> **Design intent:** *Fields read as recessed wells the user types into. The color swatch reads as a lake — a vessel filled with color rather than a button. Focus state wraps the pit rim with a gold glow ring, consistent with EFF's global focus style.*

5.3 Color Swatch

- Shape: rectangular, border-radius 8px
- Width: ~48px, height: ~36px, flex-shrink: 0
- Background: current resolved color value of the variable
- Border & shadow: see Section 5.2 — lake/vessel treatment (inset shadow only, primary border)
- On hover: cursor crosshair; transform: scale(1.06)
- Click: opens the inline color editor expand panel (Section 5.5)
- Updates in real time as the value field is edited

5.4 Name Field

- Holds the Elementor CSS variable name, e.g. --eff-color-brand-primary
- The leading "--" prefix is displayed but is not editable — protected from deletion
- Width: flex-fill — fills available space between swatch and value field
- Border & shadow: pit/recess treatment (Section 5.2)
- Directly editable — user can rename the CSS variable name portion
- Editing the name updates the CSS custom property name in the EFF data store only
- Does not write back to Elementor compiled CSS (main spec Section 14 read-only rule)
- On commit: sets hasEFFUnsavedChanges = true (Section 11.2)

5.5 Color Definition & Format Fields

Value Field:

- Width: ~180px fixed
- Border & shadow: pit/recess treatment (Section 5.2)

- Displays color value in currently selected format
- Directly editable. On valid entry: swatch updates immediately (real-time)
- On invalid entry: border turns --clr-error, background turns --clr-error-bg; swatch holds last valid color
- On commit (Enter or blur): validate and store. If invalid, restore last valid value
- Sets hasEFFUnsavedChanges = true on valid commit

Format Selector:

- Width: ~90px fixed. Dropdown styled to match EFF fields (Section 5.2 pit treatment)
- Options: HEX, HEXA, RGB, RGBA, HSL, HSLA
- Changing format converts current value and updates value field immediately — no data loss
- Format selection persisted per color variable in EFF data store

Format	Example	Notes
HEX	#2C3E50	6-digit uppercase hex, leading #
HEXA	#2C3E5080	8-digit hex with alpha
RGB	rgb(44, 62, 80)	Comma-separated, no alpha
RGBA	rgba(44, 62, 80, 0.50)	Alpha as decimal 0.00–1.00
HSL	hsl(210, 29%, 24%)	Hue degrees, S and L as percent
HSLA	hsla(210, 29%, 24%, 0.50)	Alpha as decimal 0.00–1.00

5.6 Inline Color Editor (Expand Panel)

The rightmost element of each collapsed color row is a chevron-down button (v). Clicking it opens the Inline Color Editor as an inline panel directly below the row, pushing subsequent rows downward. The chevron reverses to chevron-up (^) while the panel is open; clicking it again collapses the panel and restores the chevron-down icon. Only one expand panel may be open at a time — opening a second one closes the first.

> **Design goal:** *Full color authoring — picker, tint/shade generation, and transparency generation — without leaving the edit space. Not a modal. Opens and closes inline within the category block.*

5.6.1 Expand Panel — Full Layout Diagram

The panel is divided into three horizontal zones stacked vertically: (1) Generator Controls, (2) Preview Bars (conditional), (3) Color Picker. Each zone is separated by a 1px rule at --eff-clr-border.

```
╔═══════════════════════════════════════════════════════════════════════════╗
│  [swatch] [name──────────────────────] [value───] [fmt▼] [^]  ← ROW  ‖
╟───────────────────────────────────────────────────────────────────────────╢
│  ZONE 1 — GENERATOR CONTROLS                                            ‖
│                                                                         ‖
│  Generate tints   [ - ] [ 0 ] [ + ]     Generate shades  [ - ] [ 0 ] [ + ] ‖
│                                                                         ‖
│  Generate transparencies                 [ - ] [ 0 ] [ + ]             ‖
╟───────────────────────────────────────────────────────────────────────────╢
│  ZONE 2 — PREVIEW BARS  (each bar only visible when its count > 0)      ‖
│                                                                         ‖
│  Tints bar:   [+100][+200][+300] ... up to [+900]  (3-9 swatches)      ‖
│  Shades bar:  [-100][-200][-300] ... up to [-900]  (3-9 swatches)      ‖
│  Trans bar:   [10%][20%]...[100%]  (1-10 swatches, checkerboard bg)    ‖
╟───────────────────────────────────────────────────────────────────────────╢
│  ZONE 3 — COLOR PICKER  (always visible)                                ‖
│                                                                         ‖
│  [  2D gradient square  ]  H: [___]°  S: [___]%  L: [___]%             ‖
│  [  hue slider          ]  R: [___]   G: [___]   B: [___]              ‖
│  [  alpha slider (opt.) ]  A: [___]   HEX: [_________]                 ‖
╚═══════════════════════════════════════════════════════════════════════════╝
```

5.6.2 Zone 1 — Generator Controls

Zone 1 contains three independent counter controls, each consisting of a label, a decrement button (−), a numeric display, and an increment button (+). The controls are arranged in two rows: tints and shades side by side on the first row; transparencies on the second row.

Counter behavior (applies to all three counters):

- Default value: 0
- Tints counter range: 0, then 3–9 (skips 1 and 2 — minimum useful count is 3)
- Shades counter range: 0, then 3–9 (same rule)
- Transparencies counter range: 0–10
- Decrementing from 3 jumps directly to 0 (no values of 1 or 2 for tints/shades)
- Incrementing from 0 jumps directly to 3 for tints and shades
- At minimum (0) the − button is disabled. At maximum the + button is disabled

- Counter value is persisted per color variable in the EFF data store
- Changing any counter immediately regenerates the preview bar for that counter (Zone 2)
- Changing any counter sets hasEFFUnsavedChanges = true

Counter control styling:

- Label: --fs-sm, --eff-clr-secondary, left-aligned above or beside the counter
- − and + buttons: .eff-icon-btn style, 28×28px, gold on hover
- Numeric display: ~36px wide, bold, centered, --eff-clr-primary, no border (display only)
- Disabled − or + button: --eff-clr-muted, pointer-events: none

5.6.3 Zone 2 — Preview Bars

Zone 2 contains up to three horizontal color bars — one each for tints, shades, and transparencies. Each bar is only rendered when its counter is greater than zero. If all three counters are zero, Zone 2 is entirely absent and the panel moves directly from Zone 1 to Zone 3 with no gap.

5.6.3a Tints Bar

When the tints counter is 3–9, a horizontal bar of that many evenly-spaced tint swatches is shown, ordered lightest-to-darkest left to right (i.e. +900 on the left, +100 on the right adjacent to the main color).

- Each swatch is labeled below with its step name: +100, +200, … up to +900
- The anchor tints are fixed regardless of count — see Section 5.6.4 for the scaling algorithm
- Swatch shape: flex-fill width, 28px height, 6px border-radius, lake shadow treatment (Section 5.2)
- Clicking a swatch does not apply the color to the main variable — it is a preview only. Tint swatches are generated variables, not selectors for the main color.
- Each swatch has a small tooltip showing its resolved hex value on hover

5.6.3b Shades Bar

When the shades counter is 3–9, a horizontal bar of that many shades is shown, ordered darkest-to-lightest left to right (i.e. −900 on the left, −100 on the right adjacent to the main color). Layout and behavior identical to tints bar.

- Each swatch labeled below with its step name: −100, −200, … up to −900
- Anchor shades are fixed regardless of count — see Section 5.6.4

5.6.3c Transparencies Bar

When the transparencies counter is 1–10, a horizontal bar of that many opacity swatches is shown. Each swatch displays the main color at a specific opacity level.

- Opacity steps are evenly spaced: 1 swatch = 10% only. 2 swatches = 10%, 20%. … 10 swatches = 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%, 100%.
- Each swatch labeled below with its opacity percentage: 10%, 20%, … 100%
- Swatch background: checkerboard pattern beneath the color to make transparency visible

- Swatch shape, size, shadow: identical to tint/shade swatches
- Clicking a transparency swatch: preview only, does not change the main color value

5.6.4 Tint & Shade Scaling Algorithm

The scaling model fixes the two anchor values — the lightest tint and the darkest shade — regardless of how many steps the user selects. Intermediate steps are interpolated evenly between the anchor and the base color. This guarantees that +100 is always the first step toward white and +900 is always the near-white anchor, whether the user selects 3 steps or 9.

Tint algorithm (HSL lightness):

- Base color lightness: L (from HSL representation of the main color)
- Tint anchor (+900): fixed at L + 45% lightness (clamped to 95% maximum)
- Step size for n tints: (anchor_L − base_L) ÷ n
- +100 = base_L + 1 × step_size. +200 = base_L + 2 × step_size. … +N×100 = anchor_L
- Hue and saturation are preserved from the base color at each tint step

Shade algorithm (HSL lightness):

- Shade anchor (−900): fixed at L − 45% lightness (clamped to 5% minimum)
- Step size for n shades: (base_L − anchor_L) ÷ n
- −100 = base_L − 1 × step_size. −200 = base_L − 2 × step_size. … −N×100 = anchor_L
- Hue and saturation preserved at each shade step

Example — 3 tints vs 9 tints for a color at L = 40%:

Count	+100	+200	+300	+400	+500	+600	+700	+800	+900
3 tints	L 55%	L 70%	L 85%	—	—	—	—	—	—
5 tints	L 47.5%	L 55%	L 62.5%	L 70%	L 85%	—	—	—	—
9 tints	L 44.4%	L 48.9%	L 53.3%	L 57.8%	L 62.2%	L 66.7%	L 71.1%	L 75.6%	L 85%

> **Anchor consistency:** *+900 (the lightest tint) and −900 (the darkest shade) resolve to the same color regardless of step count. Only the intermediate steps change as the user adjusts the counter. This keeps the palette range stable while the granularity varies.*

5.6.5 Variable Naming Convention

When the user commits generated tints, shades, or transparencies to the EFF data store (by pressing a dedicated "Generate" button within Zone 1, or implicitly on Save), EFF creates named child variables derived from the main color's name. The naming follows a strict convention so EFF can recognize and re-associate them on load.

Tint names:

```
Main color name:  --eff-color-brand-primary
```

```
Tint +100:        --eff-color-brand-primary-plus-100
Tint +200:        --eff-color-brand-primary-plus-200
  ...
Tint +900:        --eff-color-brand-primary-plus-900
```

Shade names:

```
Shade -100:       --eff-color-brand-primary-minus-100
Shade -200:       --eff-color-brand-primary-minus-200
  ...
Shade -900:       --eff-color-brand-primary-minus-900
```

Transparency names:

```
Opacity 10%:      --eff-color-brand-primary-10
Opacity 20%:      --eff-color-brand-primary-20
  ...
Opacity 100%:     --eff-color-brand-primary-100
```

> **Display labels in UI:** *The color row name field shows the short suffix for readability — "+100", "−100", "10%" — with the full CSS variable name visible in the name field on hover or edit. The category row's expand button is not shown for child variables; child rows are indented beneath the parent row.*

5.6.6 Generated Variables — Data Model & Storage

Generated tint, shade, and transparency variables are full color variable records in the EFF data store. They are children of their parent variable and carry additional fields to record that relationship.

```
{
  "id":             "uuid-v4",
  "name":           "--eff-color-brand-primary-plus-100",
  "value":          "<computed hex>",
  "original_value":"<computed hex>",
  "type":           "color",
  "subtype":        "tint",          // "tint" | "shade" | "transparency"
  "parent_id":      "<uuid of main color>",
  "step":           100,             // 100-900 for tints/shades; 10-100 for transparency
  "format":         "HEX",
  "category_id":    "<same as parent>",
  "order":          2,               // follows parent in display order
  "source":         "eff-generated",
  "status":         "modified",
  "created_at":     "ISO8601",
  "updated_at":     "ISO8601"
}
```

- subtype distinguishes generated children from hand-authored variables
- parent_id links the child to its main color variable for re-association on load
- step records the numeric step value for display and naming
- source: "eff-generated" distinguishes EFF-created children from Elementor-parsed or user-created variables
- Generated variables are committed to Elementor on the next "Commit to Elementor" action along with their parent
- Deleting the parent variable: prompts "Delete [name] and its [n] generated children?" Children cannot exist without a parent — they are deleted together.

- Recalculating (changing the tint/shade counter): existing child records are updated in place. If the count decreases, surplus children are set to status "deleted". If the count increases, new children are appended.

5.6.7 Elementor Round-Trip for Generated Variables

When Sync from Elementor is run, EFF checks each parsed CSS variable name against the generated-variable naming convention. Variables whose names match the pattern [parent-name]-plus-NNN, [parent-name]-minus-NNN, or [parent-name]-NNN are treated as generated children, not as independent variables.

- If the parent exists in EFF: the child is associated with it. Its step, subtype, and tint/shade counter on the parent are reconstructed from the name.
- If no parent exists in EFF: this is a loading anomaly — see Section 12.2 for handling.
- EFF does not prevent the user from hand-editing a generated child's value. If edited, its status becomes "modified" and it is no longer recalculated when the counter changes. A small "unlinked" badge is shown on the child row.

5.6.8 Zone 3 — Color Picker

Zone 3 is always visible when the panel is open. It provides direct interactive color selection. The picker adapts its controls to the currently selected format.

Format	Picker Controls
HEX / HEXA	Hue slider + 2D Saturation/Lightness gradient square + optional Alpha slider (HEXA only). Hex input field.
RGB / RGBA	Three channel sliders R / G / B each 0–255 + optional Alpha slider (RGBA only). Numeric inputs per channel.
HSL / HSLA	Three sliders H 0–360°, S 0–100%, L 0–100% + optional Alpha slider (HSLA only). Numeric inputs per channel.

- All picker controls update the value field, swatch, and preview bars in real time
- Changing the color in the picker recalculates all tint, shade, and transparency preview swatches immediately
- Implemented in vanilla JS — no external color picker library without Jim's approval

> **Implementation note:** *Canvas-based or SVG-based picker preferred. Must work within .eff-app scope and must not conflict with WordPress admin or Elementor scripts.*

5.6.9 Panel Open / Close Behavior

- Chevron ˅ on collapsed row → opens panel, chevron becomes ^
- Chevron ^ on open panel → closes panel, chevron returns to ˅
- Opening a second panel on any other row → closes the currently open panel first, then opens the new one
- Collapsing the parent category → closes any open expand panel within it
- Panel open/close state is not persisted — all panels are closed on page load

- Animation: max-height transition 0.25s ease for open/close (see Section 9.3 CSS)

12. Elementor Load & Sync Error Handling

This section catalogues every class of anomaly that can occur when EFF reads from or writes to Elementor's color variable store, defines how each is detected, and specifies the required UI response. It supersedes the brief sync behavior notes in Section 7 for error cases.

12.1 Taxonomy of Elementor Variable Anomalies

Elementor's compiled CSS :root block may contain color variable entries that EFF cannot fully manage. These fall into four categories:

Class	Description	EFF Can Manage?
Parseable & known	Standard hex, rgb(), rgba(), hsl(), hsla() values. Known naming convention. EFF fully manages these.	Yes — full
Parseable & unknown	Valid color value but naming pattern EFF does not recognize (e.g. legacy variables, third-party plugin variables, hand-written custom names). EFF can store the value but cannot infer structure.	Partial — stored in Uncategorized
Unparseable value	Entry is a CSS custom property in the :root block but its value is not a recognizable color (e.g. a number, a dimension like "16px", a var() reference to another property, a gradient, or empty string). EFF cannot represent this as a color variable.	No — skipped with warning
Structural conflict	A variable name that EFF would generate (e.g. --eff-color-brand-primary-plus-100) exists in Elementor but its parent (--eff-color-brand-primary) does not. The child is an orphan at the naming-convention level, not just the data level.	No — requires user resolution

12.2 Load-Time Anomalies (Sync from Elementor)

When Sync from Elementor is triggered, the CSS parser runs first and produces a raw list of name/value pairs. EFF then processes each entry through the following decision tree.

12.2.1 Unparseable Value

The variable name is present in the :root block, but its value cannot be interpreted as a color by the type-inference rules (hex, rgb, rgba, hsl, hsla, named color).

- Action: Skip the variable — do not create an EFF record for it
- Logging: Add the variable name to the sync result's "skipped" list
- UI feedback: After sync completes, display a toast: "Sync complete. [n] variable(s) skipped — values EFF cannot represent as colors." A "View skipped" link opens a modal listing the names and raw values
- No user action is forced — skipped variables remain in Elementor untouched

- Example variables that trigger this: --e-global-color-accent defined as "var(--some-other-prop)", a gradient string, a pixel value, or an empty value

> **Why skip rather than warn?** *These variables are almost always non-color globals that happen to sit in the same :root block. They are not EFF's concern. Surfacing them as errors would cause noise on every sync.*

12.2.2 Orphaned Generated Child (No Parent Found)

EFF recognizes a variable name as following the generated-child convention (e.g. --eff-color-brand-primary-plus-200) but no parent variable (--eff-color-brand-primary) exists in either the current EFF data store or the parsed Elementor output.

- Action: Pause sync for this variable and display a resolution modal
- Modal title: "Unresolvable generated color"
- Modal body: "The variable [name] appears to be a generated tint/shade/transparency, but its parent color [inferred parent name] was not found in Elementor or EFF. How would you like to handle it?"
- Option A — Adopt as standalone: Create an independent EFF color record in Uncategorized. The naming convention suffix is stripped from the display label. The variable is treated as a user-created color going forward.
- Option B — Delete from Elementor: Mark the variable for deletion. It will be removed from Elementor on the next Commit to Elementor.
- Option C — Skip for now: Leave the variable in Elementor, do not create an EFF record. It will appear in the "skipped" list.
- The user's choice is applied immediately. Sync continues with the remaining variables.

12.2.3 Name Collision

Two or more variables in the parsed Elementor output have identical CSS variable names. This should not occur in valid Elementor output but can happen if CSS has been hand-edited outside Elementor.

- Action: Import the first occurrence. Flag subsequent duplicates in the sync result's "collision" list.
- UI feedback: Toast after sync — "Sync complete. [n] duplicate variable name(s) detected and skipped." Link opens a modal listing the colliding names and their values.
- The user is not forced to act — the imported version remains in EFF and the duplicate stays in Elementor unmanaged.

12.2.4 Circular var() Reference

A variable's value is a var() reference that creates a cycle (e.g. --a references --b which references --a). CSS resolves these as invalid/empty. EFF cannot resolve the color.

- Action: Skip. Add to the "skipped" list with the note "circular reference"
- No further user action required at sync time

12.3 Commit-Time Anomalies (Commit to Elementor)

When the user triggers Commit to Elementor, EFF writes its stored values back via the Elementor API. The following problems can occur during that write phase.

12.3.1 Variable Renamed in EFF — Rename Cascade

When the user renames a variable in EFF (e.g. --eff-color-brand-primary → --eff-color-brand-main), the rename must be applied atomically to the parent and all its generated children. The old names are deleted from Elementor and the new names are written in a single commit operation.

Rename cascade rules:

- The parent rename is the trigger. All child names are derived from the parent name, so they are automatically renamed to match: --eff-color-brand-primary-plus-100 → --eff-color-brand-main-plus-100, etc.
- Children whose values were manually edited (unlinked status) are renamed the same way — the suffix convention is preserved even for unlinked children
- On commit: EFF writes all new names to Elementor first, then deletes all old names. If the write phase fails for any variable, neither the writes nor the deletes are applied (full rollback)
- The pre-commit modal lists the rename cascade: "[parent name] and [n] generated children will be renamed in Elementor"

> **CRITICAL:** *CSS template references: EFF has no visibility into templates, widgets, or custom CSS blocks that reference variable names as strings. Renaming commits the rename to Elementor's global color store, but any hard-coded references to the old parent or child names will break silently. The user must audit and update template references manually. This is the highest-risk user action in EFF and must be called out prominently in the user guide.*

Rename followed by Sync (before Commit) — ghost record hazard:

If the user renames a variable in EFF and then runs Sync from Elementor before committing the rename, the following sequence occurs: EFF now holds the new name; Elementor still holds the old name. The Sync parser reads the old name from Elementor, does not find it in EFF (because EFF has the new name), and creates a new "new" status record in Uncategorized for the old name. The user now has both the renamed EFF record and a new duplicate import of the original. This is a real workflow hazard.

- Mitigation: EFF tracks a "pending rename" flag on any variable whose name has been changed but not yet committed. During Sync, if a parsed variable name matches a pending-rename original_name, EFF recognises it as the pre-commit version of a renamed variable and does not create a duplicate. Instead it updates the renamed variable's original_value as normal and leaves its new name in place.
- The pending_rename_from field is added to the variable data model (Section 6) to support this. It holds the last committed name (the name Elementor still knows). It is cleared when the rename is committed.

12.3.2 Elementor API Write Failure

The Elementor API call to write a variable fails — network error, permission error, Elementor version mismatch, or API surface change.

- Action: Abort the commit. No partial writes — if the API call fails for any variable, the entire commit is rolled back (or not applied)
- Display error modal: "Commit to Elementor failed. No changes were written. Error: [message]." with a Retry button and a Cancel button
- EFF data store is unchanged. hasPendingElementorCommit remains true
- If the failure is a permission error, add a note: "Ensure you are logged in as an Administrator and that Elementor is active."

12.3.3 Generated Child Without Parent at Commit Time

A tint, shade, or transparency child variable is queued for commit, but its parent variable has been deleted from the EFF data store since the child was generated (and the deletion has not yet been committed to Elementor).

- This state is prevented by the deletion workflow: deleting a parent variable prompts deletion of all its children (Section 5.6.6). The user must explicitly confirm.
- If somehow reached (e.g. direct data store manipulation), the pre-commit validation check catches it and reports it in the confirmation modal: "Warning: [n] generated variables have no parent. They will be committed as standalone variables."

12.3.4 Elementor Version Incompatibility

EFF targets the Elementor v4 atomic widget architecture and its specific :root block structure. If a site is running Elementor v3 (legacy) or a future version with a different CSS variable API, EFF's parser or API calls may fail entirely.

- Detection: EFF checks the Elementor version at plugin load time and again at Sync. If the version is not in the supported range, a persistent banner is shown: "EFF requires Elementor v4.x. Current version: [x.x]. Some features may not work correctly."
- Sync is still attempted in degraded mode — the parser may still find variables even if the :root structure is slightly different
- Commit to Elementor is blocked with an error if the Elementor API surface required for writing variables is not present

12.4 Variables EFF Cannot Manage

The following Elementor variable types are permanently outside EFF's scope. They are skipped silently on Sync and never appear in the EFF color list.

Variable Type	Example	Reason EFF Cannot Manage
Non-color globals	--e-global-typography-primary-font-size: 16px	Not a color. EFF's color parser ignores non-color values by design.
var() aliases	--my-color: var(--e-global-color-primary)	Value is a reference, not a resolved color. EFF cannot determine the resolved value without full CSS cascade computation.

Variable Type	Example	Reason EFF Cannot Manage
Gradient values	--bg: linear-gradient(#fff, #000)	Not a flat color. Multi-stop gradients are not representable as a single color variable.
calc() expressions	--border: calc(var(--sp) * 2)	Mathematical expression, not a color.
Empty or inherit	--x: inherit or --x: (empty)	No color value to store.
Non-EFF-prefixed colors managed by other plugins	--wc-color-primary: #e74c3c	EFF can read these but marks them source: "external". They appear in Uncategorized and are read-only — EFF will not commit changes to variables it did not originate, to avoid conflicts with other plugin managers.

> **Read-only external variables:** *Variables with source: "external" are shown in EFF with a lock icon. The user can view them and copy their values but cannot edit or commit them. This prevents EFF from accidentally overwriting variables that belong to WooCommerce, Gravity Forms, or other plugins that manage their own CSS variable sets.*

12.5 EFF-Generated Problems — Can EFF Break Elementor?

This section addresses the concern: are there actions in EFF that, when committed, could cause Elementor to malfunction or produce invalid CSS?

EFF Action	Risk to Elementor	Mitigation
Rename a variable	Any template or widget referencing the old name by string will get an unresolved variable (renders as empty / browser default)	Pre-commit modal warns explicitly. No technical block — this is a valid CSS operation.
Delete a variable	Any template or widget using the deleted variable loses its color value	Pre-commit modal lists variables to be deleted. Deleted status is staged — user must confirm twice (once in EFF, once in commit modal).
Commit an invalid hex value	Elementor stores an invalid CSS value; browser ignores it, color renders as empty	EFF validates all values before commit. Invalid values cannot reach the commit queue — the value field rejects them at entry.
Commit a generated child with a value the parent no longer matches	Tint/shade values are stale — they look wrong relative to the parent color	EFF recalculates all children whenever the parent value changes. Stale children show a "recalculate" warning badge.
Create a circular var() reference	CSS renders as invalid	EFF does not allow var() references in its value field. Only resolved color values are accepted.

EFF Action	Risk to Elementor	Mitigation
Commit during an active Elementor edit session	Elementor may overwrite EFF's committed values if the editor saves after EFF commits	EFF shows a warning if Elementor's editor is detected as open: "Elementor editor is active. Committing now may conflict with unsaved Elementor edits. Proceed?"

> **Overall assessment:** *EFF's two-store staging model (Section 11.1) is the primary safety net. Because EFF never writes to Elementor except on an explicit deliberate commit, and because all values are validated before they can enter the commit queue, the risk of EFF breaking Elementor is low. The main real-world risk is the variable rename/delete case, which is addressed by explicit pre-commit warnings.*

12.6 Comprehensive Sync & Data Integrity Hazard Register

This section catalogues every identified scenario where the interaction between EFF's data store, user actions, and Elementor's state could produce unexpected or harmful results. Each hazard is rated by likelihood and severity, and its mitigation is specified.

12.6.1 Sync Immediately After Rename (Ghost Record Hazard)

Covered in Section 12.3.1. The pending_rename_from field on the variable data model is the mitigation. Severity: High. Likelihood: Common — this is a normal workflow sequence.

12.6.2 Manual Child Edit Followed by Parent Value Change

The user opens a parent variable's expand panel, manually types a value into one of the generated child rows (e.g. edits the +200 tint directly), setting that child to "unlinked" status. Later the user changes the parent color value. The question is: does the parent value change recalculate the unlinked child or leave it alone?

- Rule: Unlinked children are never recalculated automatically. The user made a deliberate manual edit; silently overwriting it would be a data loss.
- Instead: when the parent value changes and there are unlinked children, EFF shows a non-blocking banner within the expand panel: "[n] generated child(ren) have manual edits and were not recalculated. Recalculate all? [Yes] [Keep manual edits]"
- "Yes" recalculates all children including unlinked ones, clears unlinked status. "[Keep manual edits]" leaves unlinked children untouched.
- The parent value change still sets hasPendingElementorCommit on the parent and all linked children. Unlinked children are not added to the commit queue unless their own values were changed.

12.6.3 Counter Decrease After Commit — Sync Reimports Deleted Children

The user has committed a parent with 5 tints (all 5 are live in Elementor). The user then reduces the tint counter to 3. Children +400 and +500 are set to status "deleted" in EFF but are not yet removed from Elementor (commit has not been run). The user now runs Sync before committing the deletion.

- Hazard: Sync reads Elementor, finds +400 and +500 still there. EFF has them as "deleted". The sync rule for known variables with status "deleted" must not reimport them — it must leave their "deleted" status in place.
- Rule: During Sync, variables with status "deleted" in EFF are matched against Elementor by name as usual. If found in Elementor, their status stays "deleted" and their original_value is updated to the Elementor value. They are not restored to the active variable list. They will be removed from Elementor on the next commit.
- The sync result toast notes this: "[n] variable(s) still pending deletion in Elementor."

12.6.4 Child Naming Prefix Collision

EFF identifies generated children by matching a variable name against the pattern [parent-name]-plus-NNN, [parent-name]-minus-NNN, or [parent-name]-NNN. If two parent variables share a naming prefix, the child parser could associate a child with the wrong parent.

Example: --eff-color-primary and --eff-color-primary-dark both exist. --eff-color-primary-dark-10 is a transparency child of --eff-color-primary-dark. The naive parser could read "primary" as a prefix and try to associate it with --eff-color-primary.

- Rule: The child parser must match against the longest known parent name that is a prefix of the child name. "primary-dark" is longer than "primary" and is a known parent, so --eff-color-primary-dark-10 is correctly associated with --eff-color-primary-dark.
- Implementation: At parse time, sort all known parent names by length descending and attempt matching in that order. First match wins.
- This rule must also apply when EFF generates child names: before writing a name to the store, check that the generated name does not collide with any existing parent name or any other child name.

12.6.5 Multiple EFF Files — Last Commit Wins

Nothing prevents a user from maintaining two .eff.json files for the same Elementor site (e.g. a "production" file and an "experimental" file). Loading file A and committing, then loading file B and committing, will have file B overwrite all of file A's committed values with no conflict detection, because EFF's conflict check compares only against the last Sync baseline, not against the live current Elementor state.

- Hazard severity: Medium. This is an advanced user pattern but a plausible one.
- EFF cannot fully prevent this without reading Elementor's current state at commit time and comparing it against the EFF baseline. That read-then-compare step is not currently in the commit workflow.
- Mitigation (Phase 2): Add a "verify before commit" step at the start of Commit to Elementor. EFF reads the current Elementor values, compares them against EFF's stored original_value for each variable to be committed. If any Elementor value has changed since the last Sync (i.e. another EFF file or a direct Elementor edit has updated it), EFF flags those variables as "conflict" before writing. The user can then review and resolve before the commit proceeds.
- This is the same conflict detection machinery already designed for the standard sync workflow (Section 7, status: "conflict"). Extending it to pre-commit time eliminates the multiple-file overwrite hazard.

12.6.6 EFF File Loaded on a Different Site

The user saves an .eff.json file from Site A and loads it on Site B. Site B's Elementor instance has completely different color variable values, or may not have some of the variables at all.

- At load time, EFF does not automatically sync with Elementor. The loaded file's values and statuses reflect Site A.
- When the user runs Sync on Site B, every variable whose value differs will be set to "conflict" status (since Site B's Elementor value differs from the EFF file's original_value which was Site A's value). This is a heavy conflict flood — the user will see conflict indicators on every variable.
- EFF should detect this scenario at Sync time: if the number of conflicts exceeds 50% of all synced variables, display a prominent warning modal: "Large number of conflicts detected ([n] of [total]). This may indicate the EFF file was created on a different site or against a different Elementor installation. How would you like to proceed? [Accept all Elementor values — reset to Elementor baseline] [Keep all EFF values — treat all as modified] [Review manually]"
- "Accept all Elementor values" resets all original_value fields to the just-synced Elementor values and sets all statuses to "synced". The EFF file's color values are overwritten by Elementor. This is the safe choice for a new site.
- "Keep all EFF values" sets all statuses to "modified" and proceeds. All variables will be in the commit queue. This is the right choice when deliberately migrating a palette from one site to another.

12.6.7 Undo Past a Commit — Post-Commit State Mismatch

The Undo stack is cleared on Commit to Elementor (Section 11.5). This prevents the user from undoing operations that have already been written to Elementor. However, the user may not realize the stack was cleared if they committed and then continued working.

- When the stack is cleared on commit, EFF shows a brief non-blocking toast: "Undo history cleared — Elementor is now the baseline."
- There is no undo for committed changes in Phase 2. To revert a committed change, the user must manually edit the value back in EFF and commit again. This is a known limitation noted in Section 11.5.

12.6.8 Partial Commit Rollback — API Atomicity

Section 12.3.2 specifies that a failed commit does a full rollback. However, if the Elementor API does not support transactional operations, EFF may have already written some variables to Elementor before the failure occurs. "Rollback" in that case means EFF must re-write or re-delete the variables it already changed, which is a second API operation that could itself fail.

- Implementation requirement: The eff_commit_to_elementor AJAX endpoint must track which variables were successfully written before a failure. On failure, it must attempt to restore those variables to their pre-commit values by re-issuing API write calls with the original_value. This restoration attempt is a best-effort operation.
- If restoration also fails, EFF shows an extended error modal: "Commit failed and automatic rollback was incomplete. Elementor may be in a partially updated state. Recommended action: run Sync from Elementor to re-establish the baseline, then retry your changes."
- hasPendingElementorCommit remains true. The sync result will show which variables now have "conflict" status as a result of the partial write.

12.6.9 Elementor Editor Open During Commit

If the WordPress Elementor editor is open in another browser tab while the user commits from EFF, a race condition can occur: EFF writes new values to Elementor's global color store, but the Elementor editor has already loaded the old values into its session. When the user saves in the Elementor editor, the editor's cached values overwrite EFF's committed values.

- Detection: EFF can check for an active Elementor editor session by querying a WordPress option that Elementor sets when the editor is open (elementor_edit_lock or similar). If detected, show a pre-commit warning: "The Elementor editor appears to be open. If you save in Elementor after committing, your committed changes will be overwritten. It is recommended to close the Elementor editor before committing."
- This is a warning only — the commit is not blocked, since the user may be intentionally working this way (e.g. an administrator committing while a designer uses the editor).

12.6.10 Category Moves and Elementor Sync Consistency

Categories are an EFF-only concept. Elementor has no category metadata. Moving a variable from one category to another in EFF (by drag-to-reassign or the Move action) updates category_id in the EFF data store but has no representation in Elementor.

- This is by design and is safe. Category moves set hasEFFUnsavedChanges but do not set hasPendingElementorCommit.
- On Sync, a variable's category assignment in EFF is never overwritten by Elementor — Elementor has no category data to offer. The variable's value may update, but its category stays wherever the user placed it.
- Edge case: user moves a variable to a new category in EFF, then Sync adds a new "same-named" variable from Elementor to Uncategorized (because it was brand new from Elementor's perspective). This creates a duplicate by name. This cannot happen in practice because if the variable existed in EFF (even in a different category), Sync finds it by name in the EFF store and updates it in place rather than creating a new Uncategorized entry. Category is not part of the Sync lookup key — only the CSS variable name is.

12.6.11 Variable Name Conflicts at Commit — Name Already Exists in Elementor

The user creates a brand new variable in EFF (source: "user-created") with a name that already exists in Elementor but was never imported into EFF (e.g. a variable added directly in Elementor after the last Sync). On commit, EFF writes the new variable, which silently overwrites the existing Elementor value.

- Mitigation: The "verify before commit" pre-commit Elementor read (introduced in Section 12.6.5) catches this. If a variable to be committed has source: "user-created" and EFF finds a current Elementor value for that name that EFF has no record of, it flags it: "New variable [name] conflicts with an existing Elementor variable not in EFF. Elementor value: [value]. EFF value: [value]. Overwrite Elementor? [Yes] [Cancel]"

12.6.12 Summary — Data Integrity Rules

The following rules, taken together, form the complete data integrity contract for the EFF two-store model:

Rule	Description
Pending rename tracking	pending_rename_from is set on rename, cleared on commit. Sync uses it to avoid ghost records.
Longest-prefix child matching	Child-to-parent association always matches the longest known parent name first.
Deleted variables survive Sync	Status "deleted" is preserved through Sync. Deleted variables are not reimported.
Unlinked children not auto-recalculated	Manual child edits survive parent value changes. User is prompted to choose.
Pre-commit Elementor read	Before writing, EFF reads current Elementor values and flags any unexpected changes as conflicts.
Atomic commit with best-effort rollback	On commit failure, EFF attempts to restore already-written variables. On restoration failure, user is directed to Sync.
Category moves never touch Elementor	Category assignment is EFF-only metadata. It is never written to or read from Elementor.
No undo past commit	Undo stack clears on commit. Reversion requires a new manual edit + commit cycle.
Large conflict flood detection	If >50% of variables conflict on Sync, EFF offers bulk resolution options to handle file-migration scenarios.
External variables are read-only	Variables with source: "external" cannot be committed. They are display-only in EFF.

6. Color Variable Data Model

```
{
  "id":                   "uuid-v4",
  "name":                 "--eff-color-brand-primary",
  "value":                "#2C3E50",
  "original_value":       "#2C3E50",
  "pending_rename_from": null,
  "type":                 "color",
  "format":               "HEX",
  "group":                "Variables",
  "subgroup":             "Colors",
  "category_id":          "uuid-v4",
  "category":             "Branding",
  "order":                1,
  "source":               "elementor-parsed",
  "status":               "synced",
  "created_at":           "ISO8601",
  "updated_at":           "ISO8601"
}
```

Field	Type	Description
original_value	string	The value as last read from Elementor's compiled CSS. Stored at parse time and updated only on Sync. Used to detect user modifications and conflicts.
pending_rename_from	string|null	The name last committed to Elementor. Set when the user renames a variable; cleared when the rename is committed. Used during Sync to prevent ghost-record creation (Section 12.3.1).
format	string	Active display/edit format. One of: HEX, HEXA, RGB, RGBA, HSL, HSLA. Default: HEX.
category_id	uuid	Foreign key to the category this variable belongs to.
category	string	Denormalized category name for display convenience.
order	integer	Display order within the category (1-based).
status	string	One of: "synced", "modified", "conflict", "orphaned", "new", "deleted". See Section 7. Replaces the simple boolean "modified" field.

> **status replaces modified:** *The original boolean "modified" field is replaced by the richer "status" enum. This directly addresses the question of tracking deletions and other states — see Section 6.1.*

6.1 Status Field Values

Status	Meaning	UI Indicator
synced	Value matches Elementor original. No user changes.	None (clean state)
modified	User has changed the value from the Elementor original.	Pencil icon — gold (--eff-clr-accent)
conflict	User has modified value AND Elementor has also changed it since last sync.	Warning triangle — amber (--clr-warning)

Status	Meaning	UI Indicator
orphaned	Variable exists in EFF but is no longer present in Elementor's output.	Broken chain icon — red (--clr-error)
new	Variable appeared in Elementor output since last sync, just added to Uncategorized.	Sparkle badge — blue (--clr-info)
deleted	User has marked this variable for deletion within EFF. Not yet committed.	Strike-through row + trash icon — muted

> **On "deleted" status:** *When a user deletes a color variable row, its status is set to "deleted" and the row is visually struck through and dimmed, rather than immediately removed from the data store. The row is fully removed when the user saves the EFF file (or committed to Elementor). This provides a lightweight undo within the session — the user can un-delete before saving.*

7. Sync Behavior (Colors)

Sync reads Elementor's compiled CSS and updates EFF's internal data store. It does not push data back to Elementor — that is the Commit operation (Section 11.3). Because EFF and Elementor are updated independently, a genuine conflict (both changed simultaneously) is not a normal use case — see Section 11.4 for the full analysis.

1. Parse the Elementor v4 :root block and extract all color variables (type-inferred: hex, rgb(), rgba(), hsl(), hsla(), CSS named colors)
2. For each parsed variable, look up EFF data store by CSS variable name
3. Known variable with status "synced": update stored value and original_value to match Elementor. Status remains "synced".
4. Known variable with status "modified": compare Elementor's new value against original_value. If Elementor value unchanged: leave EFF value alone, status stays "modified". If Elementor value also changed: set status to "conflict" and show warning indicator
5. Unknown variable (not in EFF store): add to Uncategorized with status "new", source "elementor-parsed"
6. EFF variable absent from parsed output: set status to "orphaned" — do not delete automatically

State	Indicator Icon	Color Token
synced	None	—
modified	Pencil on swatch	--eff-clr-accent (#f4c542)
conflict	Warning triangle	--clr-warning (#f59e0b)
orphaned	Broken chain	--clr-error (#dc2626)
new	Sparkle badge	--clr-info (#3b82f6)
deleted	Strike-through row	--eff-clr-muted (#64748b)

8. Interactions & State

> **hasUnsavedChanges clarification:** *The flag "hasUnsavedChanges" in this and earlier sections now means "EFF data store has unsaved changes" — i.e., the EFF .eff.json file is out of date. It has no relation to Elementor. Drag-and-drop, renaming, reordering, and color edits all set this flag. See Section 11.2 for the full state model.*

8.1 Drag-to-Reassign

- Visual drag handle: grip icon (six dots) on far left of each row, visible on hover
- Drag target zones: above or below any row, or into an empty/collapsed category header
- Drag to different category: updates category_id and category name in EFF data store
- Drag within same category: updates order values
- All drag operations: set hasEFFUnsavedChanges = true

8.2 Add Color Row

- + button in category header appends new empty row at bottom of that category
- Name: empty, focused immediately. Value: #000000. Format: HEX. Source: "user-created". Status: "modified".
- Sets hasEFFUnsavedChanges = true

8.3 Delete Color Row

- Trash icon appears on row hover (far right, after expand button)
- Click: inline micro-confirmation "Delete [name]? [Yes] [No]" — not a full modal
- On confirm: sets status to "deleted", row visually struck through and dimmed
- Full removal from data store on next EFF file save or Commit to Elementor
- Sets hasEFFUnsavedChanges = true

8.4 Filter & Search

- Filter bar at top of Colors page — real-time substring match on name or value
- Category headers remain visible; empty categories hidden while filter is active
- Clearing filter restores all rows and categories
- Filter state not persisted — resets on navigation away from Colors page

8.5 Collapse All / Expand All

- "Collapse all" and "Expand all" text links at top-right of filter bar
- Lightweight text links — --fs-sm, --eff-clr-link (not primary buttons)

8.6 Undo / Redo

See Section 11.5 for the full Undo/Redo architecture. Summary:

- Full Undo/Redo stack for all EFF data store operations within a session
- Keyboard shortcuts: Ctrl+Z (Undo), Ctrl+Y / Ctrl+Shift+Z (Redo)
- Undo/Redo buttons also available in the top menu bar (History icon)
- Undo/Redo stack is cleared when a Commit to Elementor is executed — at that point the Elementor state is the new baseline

9. CSS & Styling Details

9.1 Category Header CSS

```
.eff-category-header {
  display:         flex;
  align-items:     center;
  justify-content: space-between;
  padding:         0 var(--sp-3);
  height:          40px;
  background:      var(--eff-bg-panel);
  border-radius:   6px 6px 0 0;
  border-bottom:   1px solid var(--eff-clr-border);
}
.eff-category-header.is-collapsed {
  border-radius: 6px;
  border-bottom: none;
}
.eff-category-name  { font-size: var(--fs-md); font-weight: var(--fw-semibold); color: var(--
eff-clr-primary); }
.eff-category-dot   { width: 8px; height: 8px; border-radius: 50%; background: var(--eff-clr-
accent); }
```

9.2 Color Row CSS

```
.eff-color-row {
  display:       flex;
  align-items:   center;
  gap:           var(--sp-2);
  padding:       var(--sp-2) var(--sp-3);
  min-height:    44px;
  border-bottom: 1px solid var(--eff-clr-border);
  transition:    background 0.1s ease;
}
.eff-color-row:nth-child(even) { background: var(--eff-bg-card);  }
.eff-color-row:nth-child(odd)  { background: var(--eff-bg-panel); }
.eff-color-row:hover           { background: rgba(244,197,66,0.06); }
.eff-color-row.is-deleted      { opacity: 0.45; text-decoration: line-through; }

.eff-color-field       { height: 36px; border-radius: 8px; padding: 0 10px;
                         font-size: var(--fs-sm); background: var(--eff-bg-field); }
.eff-color-field.name  { flex: 1; }
.eff-color-field.value { width: 180px; flex-shrink: 0; font-family: "Courier New", monospace; }
.eff-color-format      { width: 90px; flex-shrink: 0; height: 36px; border-radius: 8px; }

/* Pit/recess field borders — see Section 5.2 for token definitions */
.eff-color-field, .eff-color-format {
  border:     1px solid var(--eff-clr-primary);
  box-shadow: var(--eff-field-shadow);
}
.eff-color-field.invalid { border-color: var(--clr-error); background: var(--clr-error-bg); }

/* Swatch lake — see Section 5.2 */
.eff-color-swatch {
  width: 48px; height: 36px; border-radius: 8px; flex-shrink: 0; cursor: crosshair;
  border:     1px solid var(--eff-clr-primary);
  box-shadow: var(--eff-swatch-shadow);
}
```

9.3 Expand Panel CSS

```
.eff-color-expand-panel {
  background:    var(--eff-bg-card);
  border:        1px solid var(--eff-clr-border);
```

```
  border-top:    none;
  border-radius: 0 0 8px 8px;
  overflow:      hidden;
  max-height:    0;
  transition:    max-height 0.25s ease, padding 0.25s ease;
}
.eff-color-expand-panel.is-open { max-height: 600px; padding: var(--sp-4); }

.eff-tints-strip  { display: flex; gap: var(--sp-1); margin-bottom: var(--sp-4); }
.eff-tint-swatch  { flex: 1; height: 28px; border-radius: 6px; cursor: pointer; }
.eff-tint-swatch.is-base { border: 2px solid var(--eff-clr-accent); }
```

10. Phase 2 Implementation Checklist

10.1 Data Layer

- Update eff-defaults.json: rename "Status" → "Semantic", add "Uncategorized" as 5th category
- Replace "modified" boolean with "status" enum in variable data model
- Add "original_value" field to variable data model
- Add two-store architecture: eff_data (EFF store) + eff_elementor_baseline (last synced Elementor values)
- Implement Undo/Redo command stack in JS (Section 11.5)
- Add category CRUD to class-eff-data-store.php
- Add color variable CRUD and reorder methods
- Add EFF clipboard store (in-memory JS, not persisted)

10.2 CSS Parser

- Add color type inference: detect hex, rgb(), rgba(), hsl(), hsla(), named colors
- Implement Sync logic per Section 7: synced/modified/conflict/orphaned/new status updates
- Return sync result summary { added, updated, conflicted, orphaned } for UI feedback toast

10.3 Left Menu

- Dynamically render color category subgroups from EFF data store
- Real-time sync between edit space category operations and left menu

10.4 Colors Page — UI

- Build eff-colors-page.js
- Render category blocks with headers and color rows
- Category header: all 8 icon buttons with correct disabled states
- Inline category name editing
- Color row: swatch (lake shadow), name field, value field, format selector, expand button (all pit/recess borders per Section 5.2)
- Real-time swatch updates on value field edit
- Format conversion (HEX ↔ RGB ↔ HSL) on format selector change
- Field validation with error state styling
- Expand panel (Section 5.6): Zone 1 generator controls (tints 0/3–9, shades 0/3–9, transparencies 0–10), Zone 2 conditional preview bars, Zone 3 color picker
- Tint/shade scaling algorithm per Section 5.6.4 — fixed anchors, interpolated steps
- Transparency bar with checkerboard background
- Generated variable naming convention (Section 5.6.5) and child data records (Section 5.6.6)
- Drag-to-reorder within category; drag-to-reassign between categories
- Row delete with "deleted" status + struck-through row; undo via Undo stack
- Filter/search bar with real-time filtering
- Collapse All / Expand All controls
- Sync status indicators: modified, conflict, orphaned, new, deleted
- Undo/Redo keyboard shortcuts and top-bar buttons

- "Commit to Elementor" button — separate from "Save EFF File" (Section 11.3)

10.5 AJAX

- eff_save_category, eff_delete_category, eff_reorder_categories
- eff_save_color, eff_delete_color, eff_reorder_colors
- eff_generate_children — creates/updates tint, shade, transparency child records for a parent variable
- eff_commit_to_elementor — writes EFF store values back via Elementor API
- All endpoints protected with check_ajax_referer()

11. Architecture Decisions & Open Questions

This section records the architectural decisions reached in response to Jim's review questions. It is a permanent part of the spec — Claude Code must read and implement accordingly.

11.1 The Two-Store Model (Answer to Questions 1, 4, 5, 6)

EFF operates as a deliberate staging layer between the user and Elementor. There are two distinct data stores:

Store	What it holds	When it changes
EFF Data Store	All variables, categories, names, formats, order — everything the user edits in EFF.	On every user edit (name change, color change, drag, add, delete). Persisted to .eff.json on "Save EFF File".
Elementor Baseline	The last-known values as read from Elementor's compiled CSS. Read-only mirror.	Updated only when "Sync from Elementor" is triggered. Never written by user edits.

Elementor's own CSS file is never touched by EFF during normal editing. EFF values are pushed back to Elementor only when the user explicitly triggers "Commit to Elementor" (Section 11.3). This means:

- The user can freely experiment in EFF, undo, rearrange, and rename without any risk to the live Elementor configuration
- The user decides when to push changes to Elementor — it is a deliberate, separate action
- There is no scenario where both EFF and Elementor are editing the same variable simultaneously during a normal session, which eliminates the conflict case Jim identified in Question 4

> **Architectural decision:** *EFF is a staging layer. User edits affect EFF only. Elementor is updated only on explicit "Commit to Elementor". This is the safest model and is the one specified throughout this document.*

11.2 Save State Flags (Answer to Question 5)

There are now two distinct save-state flags, not one:

Flag	Meaning	What sets it	What clears it
hasEFFUnsavedChanges	The EFF .eff.json file is out of date with the in-memory EFF data store.	Any edit to EFF data: color values, names, drag/drop, add, delete, rename, reorder.	"Save EFF File" — writes .eff.json to disk via AJAX.
hasPendingElementorCommit	EFF data store has changes that have not yet been pushed to Elementor.	Any edit to a variable's value, name, or deletion (not structural EFF-only changes like category reorder).	"Commit to Elementor" — pushes values via Elementor API.

The "Save Changes" button on the right panel of the main display corresponds to hasEFFUnsavedChanges — it saves the EFF file. Drag-and-drop reordering correctly sets hasEFFUnsavedChanges but does not set hasPendingElementorCommit, because category order is not an Elementor concept.

> **Right panel button:** *The existing "Save Changes" button saves the EFF data store to the .eff.json file. It does not push to Elementor. A separate "Commit to Elementor" action is needed (Section 11.3).*

11.3 Commit to Elementor Action (Answer to Questions 1 & 6)

"Commit to Elementor" is the explicit action that writes EFF's variable values back into Elementor via the Elementor API. It is separate from saving the EFF file.

- Location: top menu bar — a dedicated button (icon: send/upload arrow, tooltip: "Commit to Elementor")
- Enabled only when hasPendingElementorCommit = true
- Before executing: shows a confirmation modal listing the number of variables to be committed and any that are in "conflict" or "orphaned" state
- On confirmation: calls eff_commit_to_elementor AJAX endpoint, which uses the Elementor API to write variable values to Elementor's data
- On success: updates all committed variables' original_value fields to match current value. Sets all committed variable statuses to "synced". Clears hasPendingElementorCommit. Clears Undo/Redo stack (the committed state is now the new baseline)
- On failure: displays error toast. EFF data store unchanged. User can retry

> **CRITICAL:** *Commit to Elementor uses the Elementor API, not direct CSS file editing. EFF never writes to Elementor's compiled CSS output directly. This is consistent with main spec Section 14. The specific Elementor v4 API surface for writing variables must be researched before implementing this endpoint.*

11.4 Conflict Case Analysis (Answer to Question 4)

Jim correctly identified that in the two-store model, the conflict scenario (both EFF and Elementor modified simultaneously) has no realistic use case in normal operation. Here is the analysis:

- Normal workflow: User edits colors in EFF → saves EFF file → commits to Elementor. Elementor is only updated when the user says so. No simultaneous editing.
- The "conflict" status (Section 6.1) would only arise if: (1) someone edits the Elementor variables directly in Elementor (not through EFF), AND (2) the user has also modified that same variable in EFF, AND (3) the user then triggers Sync. This is an edge case, not the primary workflow.
- The conflict detection machinery is retained in the spec as a safety net for that edge case. It is not expected to fire in normal use.

> **Decision:** *Conflict detection is kept as a defensive feature. Its UI indicators are low-cost. The primary workflow has no conflicts.*

11.5 Undo / Redo Architecture (Answer to Question 6)

EFF implements a command-pattern Undo/Redo stack for all EFF data store operations.

What is undoable:

- Color value changes
- Variable name changes
- Format selector changes
- Category renames, reorder, add, delete
- Variable add, delete (restores from "deleted" status), drag-to-reassign

What is not undoable:

- Sync from Elementor (this is a read operation; it updates the baseline, not the user's edits)
- Save EFF File (file was written; Undo does not unwrite files)
- Commit to Elementor — once committed, the Undo stack is cleared. Elementor is now the new baseline.

Implementation:

- JS command stack: array of { do, undo } function pairs
- Every user-initiated data change pushes a command onto the stack
- Ctrl+Z executes undo() of the top command and moves the pointer back
- Ctrl+Y / Ctrl+Shift+Z executes redo() and moves the pointer forward
- Stack limit: 50 operations (configurable in Preferences)
- Undo/Redo buttons in the top menu bar show the operation name on hover tooltip (e.g., "Undo: renamed --eff-color-brand-primary")

> **Post-commit undo:** *Undoing after a Commit to Elementor would require reverting Elementor — which EFF does not support in Phase 2. The stack is cleared on commit to prevent confusion. A future History/undo phase (v5 in the roadmap) could address post-commit history.*

11.6 Field Border — Resolved (Answer to Question 2)

Jim reviewed eff-field-border-sample.html (v2), which presented three pit/recess depth candidates alongside the original flat style. Variant A — Subtle Pit — was approved.

The approved CSS tokens are now final in Section 5.2. The shadow model is a pit/recess effect: dark outer and inner shadows on the near edges (top, left), warm/light shadows on the far edges (bottom, right). The swatch uses inset-only shadow on all four walls — a lake or vessel filled with color. Section 5.2 requires no further review.

> **Resolved:** *Variant A (Subtle Pit) approved by Jim Roberts. Section 5.2 tokens are final. Add --eff-field-shadow and --eff-swatch-shadow to eff-theme.css at implementation time.*

End of EFF Phase 2 — Colors Variables Specification v2.0

© Jim Roberts / Jim R Forge — jimrforge.com